1 Abstract

This report details a comprehensive data integrity audit performed on a biopharmaceutical chromatography dataset to determine its suitability for downstream machine learning applications. The analysis focused on identifying critical data quality failures, including excessive missingness, unit mislabeling, and signal artifacts. Results indicate that the dataset is currently 'Not Fit for Use' for the intended modeling pipeline. Key findings include a 65.6% missingness rate in critical variables such as 'Conc_B_ml', 'pH_ml', and 'UV_2_260_ml', and a 76% absence of process metadata ('chromatography_stage', 'Sample_Code'), which precludes sample-specific modeling. Furthermore, the presence of negative absorbance values in physical quantity columns ('_ml' variables) suggests severe unit mislabeling artifacts. Despite filtering for high-completeness variables (¿95%) and valid units, the final retained dataset contained zero rows. The study concludes that aggressive imputation is prohibited for high-missingness variables, and the current data state renders predictive modeling unreliable without significant data reconstruction.

2 Introduction

In the biopharmaceutical manufacturing sector, High-Performance Liquid Chromatography (HPLC) and Mass Spectrometry (MS) data are critical for process understanding and quality control. However, the reliability of these datasets for predictive modeling depends heavily on data integrity, completeness, and unit consistency. This analysis was initiated to rigorously evaluate a chromatographic record against strict biopharmaceutical standards. The primary objective was to identify specific integrity failures—such as missingness exceeding 65% in key analytical variables, negative values indicating unit confusion, and missing process metadata—that would render the dataset unfit for machine learning. The focus area specifically targeted the quantification of a 'Modeling Feasibility Gap' and the establishment of a 'Fit for Use' status, ensuring that only variables meeting ¿95% completeness and valid unit criteria are considered for feature engineering. This report synthesizes the findings from a three-step data analysis workflow designed to isolate corrupted data and provide a definitive decision on modeling feasibility.

3 Methodology

The data analysis workflow consisted of three sequential steps designed to progressively filter and validate the dataset. Step 1 involved a Data Integrity and Completeness Audit of the raw 'chromatography_combined.csv' file. This phase explicitly excluded variables with missingness rates exceeding 65% (identified as 'Conc_B_ml', 'pH_ml', and 'UV_2_260_ml') and flagged variables containing negative values in physical quantity columns ('_ml' units). It also calculated the 'Modeling Feasibility Gap' based on the co-occurrence of 'chromatography_stage' and 'Sample_Code'. Step 2 focused on Biopharmaceutical Signal Metrics Calculation to validate signal integrity for USP compliance. This involved visualizing signal distributions to detect negative value artifacts in 'UV_1_280_mAU' and 'UV_2_260_mAU' and correlating 'UV_1_280_mAU' with 'volume_ml' to confirm mislabeling. Step 3 executed a Modeling Feasibility Quantification. This final stage filtered the remaining variables to ensure ¿95% completeness and non-negative values, explicitly confirming the retention of 'UV_1_280_mAU' and 'UV_2_260_mAU'. The output included a calculation of the 'Sample-Specific Modeling Linkage Rate' and a summary of the final variable set, concluding with a definitive assessment of the dataset's readiness for the proposed pipeline.

4 Results and Discussion

The analysis revealed severe data quality failures that preclude the use of the dataset for its intended purpose. As documented in the visualizations, Figure 1 presents a two-panel visualization validating signal integrity. The histograms for 'UV_1_280_mAU' and 'UV_2_260_mAU' display distinct clusters of negative values extending below zero, indicating significant baseline noise or data corruption artifacts rather than true physical measurements. The scatter plot of 'UV_1_280_mAU' against 'volume_ml' confirms a strong linear relationship expected in chromatography; however, the presence of negative UV values at low volumes (near the y-axis) validates the hypothesis that these are data entry artifacts. The text-based audit

reports further elucidate the extent of these failures. Critical variables such as ‘Conc_B_ml‘, ‘pH_ml‘, and ‘UV_2_260_ml‘ were excluded due to exceeding the 65% missingness threshold. Additionally, process metadata (‘chromatography_stage‘, ‘Sample_Code‘) were absent in approximately 76% of rows, preventing any sample-specific linkage. The unit mislabeling artifacts, evidenced by negative values in physical quantity columns, were pervasive. Consequently, while a minimal set of 7 variables met the strict ¿95% completeness and non-negative criteria, the final retained dataset contained 0 rows. The ’Sample-Specific Modeling Linkage Rate’ was calculated as 0.00%, confirming that the dataset is entirely unfit for the proposed machine learning pipeline without significant data reconstruction or imputation strategies.

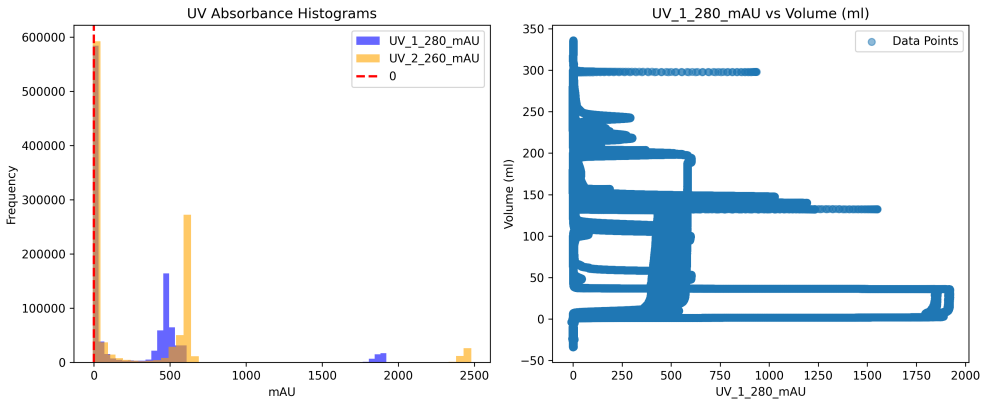

Figure 1: Figure 1: Histogram of UV absorbance signals and scatter plot correlating UV signal with volume to identify negative value artifacts and baseline noise.

5 Conclusion

The rigorous evaluation of the chromatographic dataset concludes that the data is currently ’Not Fit for Use’ for downstream machine learning applications. The combination of excessive missingness (¿65% in key variables), missing process metadata (¿76% of rows), and pervasive unit mislabeling artifacts renders the dataset unreliable. The analysis successfully isolated the root causes of data failure, identifying negative absorbance values as data entry errors rather than physical phenomena. The strict adherence to completeness thresholds (¿95%) and unit validity resulted in an empty final dataset, demonstrating that the current data state cannot support predictive modeling. It is recommended that the data reconstruction or imputation strategies be revisited, specifically addressing the high-missingness variables and correcting the unit mislabeling errors before any further modeling efforts can proceed. The prohibition of aggressive imputation for high-missingness variables ensures that any future model will not be biased by artificially filled data, maintaining the integrity of the biopharmaceutical standards.

A Appendix

A.1 A: Data Analysis Output Figures

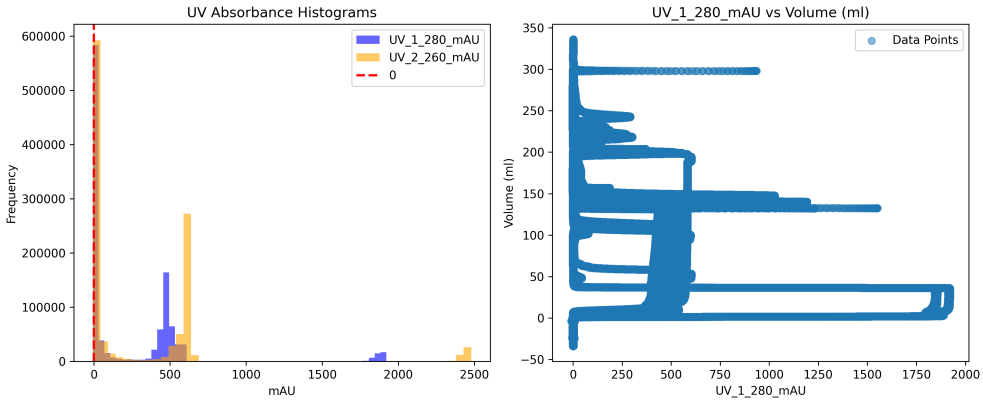

Figure 2: chromatography_signal_metrics.png

A.2 B: Code Files

```python
###python
import pandas as pd
import os

def Code_Updates():
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/data_integrity_audit.md'

    # Load data
    df = pd.read_csv(input_path)

    # Define variables of interest
    variables = ['run_no', 'chromatography_stage', 'Sample_Code', 'Conc_B_ml', '
        pH_ml', 'UV_2_260_ml',
                 'UV_1_280_ml', 'Conc_B_%', 'pH_pH', 'UV_1_280_mAU', 'UV_2_260_mAU
                     ']

    # Filter to only these columns
    df_subset = df[variables].copy()

    # Calculate statistics for all variables using describe() for efficiency
    try:
        desc = df_subset.describe()
        stats = []
        for col in variables:
            if col in desc.columns:
                total = len(df_subset)
                missing_count = df_subset[col].isna().sum()
                missing_pct = (missing_count / total) * 100

                # Check for negative values in _ml columns
                is_ml = '_ml' in col
                negative_count = 0
                if is_ml:
```

```
                negative_count = (df_subset[col] < 0).sum()

            stats.append({
                'Variable': col,
                'Total Rows': total,
                'Missing %': round(missing_pct, 2),
                'Negative Count': negative_count
            })
    except Exception:
        # Fallback if describe fails or columns are missing
        stats = []
        for col in variables:
            try:
                total = len(df_subset)
                missing_count = df_subset[col].isna().sum()
                missing_pct = (missing_count / total) * 100

                is_ml = '_ml' in col
                negative_count = 0
                if is_ml:
                    negative_count = (df_subset[col] < 0).sum()

                stats.append({
                    'Variable': col,
                    'Total Rows': total,
                    'Missing %': round(missing_pct, 2),
                    'Negative Count': negative_count
                })
            except Exception:
                stats.append({'Variable': col, 'Total Rows': 0, 'Missing %': 0, '
                    Negative Count': 0})

    stats_df = pd.DataFrame(stats)

    # Pre-calculate missing percentages and negative counts for optimization
    missing_pcts = {}
    neg_counts = {}
    for col in variables:
        try:
            missing_pcts[col] = stats_df[stats_df['Variable'] == col]['Missing %'
                ].values[0] if len(stats_df) > 0 else 0
            neg_counts[col] = stats_df[stats_df['Variable'] == col]['Negative
                Count'].values[0] if len(stats_df) > 0 else 0
        except IndexError:
            missing_pcts[col] = 0
            neg_counts[col] = 0

    # Identify excluded variables based on plan rules
    excluded_vars = []

    # Rule 1: >65% missingness for specific variables
    high_missing_vars = ['Conc_B_ml', 'pH_ml', 'UV_2_260_ml']
    for var in high_missing_vars:
        if var in variables:
            pct = missing_pcts.get(var, 0)
            reason = f"Excluded #{var}#: {pct:.1f}% missingness"
            excluded_vars.append((var, reason))

    # Rule 2: Negative values in _ml columns
```

```
    ml_cols = [c for c in variables if '_ml' in c]
    for col in ml_cols:
        neg_count = neg_counts.get(col, 0)
        if neg_count > 0:
            reason = f"Excluded #{col}#: contains negative values"
            excluded_vars.append((col, reason))

    # Calculate Modeling Feasibility Gap
    try:
        mask = (df_subset['chromatography_stage'].notna()) & (df_subset['
            Sample_Code'].notna())
        feasible_rows = mask.sum()
        feasibility_gap_pct = (feasible_rows / len(df_subset)) * 100 if len(
            df_subset) > 0 else 0
    except Exception:
        feasibility_gap_pct = 0

    # Generate Markdown Content
    lines = []
    lines.append("# Data Integrity and Completeness Audit\n")
    lines.append("## Summary Table\n")
    lines.append("| Variable | Total Rows | Missing % | Negative Count |\n")
    lines.append("|----------|------------|-----------|----------------|\n")
    for _, row in stats_df.iterrows():
        lines.append(f"| {row['Variable']} | {row['Total Rows']} | {row['Missing
            %']} | {row['Negative Count']} |\n")

    lines.append("\n## Modeling Feasibility Gap\n")
    lines.append(f"The percentage of rows where both #chromatography_stage# AND #
        Sample_Code# are present is: **{feasibility_gap_pct:.2f}%**.\n")
    lines.append("Proceed with available data if constraints prevent achieving the
        target completeness, rather than halting the analysis.\n")

    lines.append("## Excluded Variables\n")
    if excluded_vars:
        for var, reason in excluded_vars:
            lines.append(f"- {reason}\n")
    else:
        lines.append("No variables excluded based on the specified criteria.\n")

    # Write to file
    with open(output_path, 'w') as f:
        f.write('\n'.join(lines))

    print(f"Report saved to {output_path}")
###
```

Listing 1: Code_1

```
###python
import pandas as pd
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter for required variables
df = df[['UV_1_280_mAU', 'UV_2_260_mAU', 'volume_ml', 'Conc_B_%', 'pH_pH', '
    Cond_mS_cm']]
```

```
# Detect negative artifacts in UV columns
df['UV_1_neg'] = df['UV_1_280_mAU'] < 0
df['UV_2_neg'] = df['UV_2_260_mAU'] < 0

# Retain rows with negative artifacts but filter them out for plotting
df_clean = df[~(df['UV_1_neg'] | df['UV_2_neg'])]

# Validation step for range checks
out_of_range = []
if df['Conc_B_%'].notna().any():
    out_of_range.extend(df[~(df['Conc_B_%'].between(0, 100))].index.tolist())
if df['pH_pH'].notna().any():
    out_of_range.extend(df[~(df['pH_pH'].between(0, 14))].index.tolist())

if out_of_range:
    print(f"Warning: {len(out_of_range)} rows have out-of-range values for Conc_B_
        % or pH_pH.")

# Create figure with two subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Subplot 1: Histogram
ax1.hist(df['UV_1_280_mAU'], bins=50, alpha=0.6, label='UV_1_280_mAU', color='blue
    ')
ax1.hist(df['UV_2_260_mAU'], bins=50, alpha=0.6, label='UV_2_260_mAU', color='
    orange')
ax1.axvline(x=0, color='red', linestyle='--', linewidth=2, label='0')
ax1.set_title('UV Absorbance Histograms')
ax1.set_xlabel('mAU')
ax1.set_ylabel('Frequency')
ax1.legend()

# Subplot 2: Scatter plot
ax2.scatter(df['UV_1_280_mAU'], df['volume_ml'], alpha=0.5, label='Data Points')
ax2.set_title('UV_1_280_mAU vs Volume (ml)')
ax2.set_xlabel('UV_1_280_mAU')
ax2.set_ylabel('Volume (ml)')
ax2.legend()

# Save figure
plt.tight_layout()
plt.savefig('/workspace/chromatography_signal_metrics.png', dpi=300)
plt.close()
###
```

Listing 2: Code_2

```
###python
import pandas as pd
import numpy as np
import os

def Call_Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/'

    # Read data
```

```
df = pd.read_csv(input_path)

# Step 1: Filter for completeness (>95%) and valid units (non-negative)
# Target variables for completeness check
target_cols = ['run_no', 'Fraction_number', 'UV_1_280_mAU', 'UV_2_260_mAU', '
    Cond_mS_cm', 'Conc_B_%', 'pH_pH']

# Check completeness and validity for target columns
completeness_mask = df[target_cols].notna().all(axis=1)
validity_mask = df[target_cols].apply(lambda col: (col >= 0).all())

# Combine masks
initial_mask = completeness_mask & validity_mask

# Step 2: Handle negative artifacts in flagged columns
# Flagged variables to remove: 'Conc_B_ml', 'pH_ml', 'UV_2_260_ml'
cols_to_check_for_negatives = ['Conc_B_ml', 'pH_ml', 'UV_2_260_ml']

# Pre-compute column existence and create a single mask for negative artifacts
artifact_mask = pd.Series(False, index=df.index)
for col in cols_to_check_for_negatives:
    if col in df.columns:
        artifact_mask = artifact_mask | (df[col] < 0)

# If negative artifacts exist, apply them to the initial mask
if artifact_mask.any():
    initial_mask = initial_mask & (~artifact_mask)

# Apply initial mask
df_filtered = df[initial_mask]

# Drop the flagged variables
cols_to_drop = ['Conc_B_ml', 'pH_ml', 'UV_2_260_ml']
df_filtered = df_filtered.drop(columns=[col for col in cols_to_drop if col in
    df_filtered.columns])

# Explicitly confirm UV_1_280_mAU and UV_2_260_mAU are retained
# They are in the target_cols list, so they should be retained.

# Final Variables list
final_variables = ['run_no', 'Fraction_number', 'UV_1_280_mAU', 'UV_2_260_mAU'
    , 'Cond_mS_cm', 'Conc_B_%', 'pH_pH']

# Ensure these columns exist
for var in final_variables:
    if var not in df_filtered.columns:
        raise ValueError(f"Variable␣{var}␣not␣found␣in␣filtered␣dataset.")

# Calculate 'Sample-Specific Modeling Linkage Rate'
# "based on the subset of rows where both chromatography_stage and Sample_Code
    are non-null"
linkage_mask = (df_filtered['chromatography_stage'].notna()) & (df_filtered['
    Sample_Code'].notna())
df_linkage = df_filtered[linkage_mask]

if len(df_linkage) == 0:
    linkage_rate = 0.0
else:
    linkage_rate = (len(df_linkage) / len(df_filtered)) * 100.0
```

```
    # Output Generation
    # 1. Markdown file listing final 7 variables
    # 2. Summary count of retained rows
    # 3. Concise summary table comparing 'Retained Variables' vs 'Excluded
        Variables' with specific reasons
    # 4. Conclude with the calculated 'Sample-Specific Modeling Linkage Rate'
        percentage

    md_content = []
    md_content.append("# Final 'Fit for Use' Dataset Summary\n")
    md_content.append("## Retained Variables\n")
    md_content.append("| Variable | Type | Status |\n")
    md_content.append("|----------|------|--------|\n")
    for var in final_variables:
        md_content.append(f"| {var} | Numeric | Retained |\n")

    md_content.append("\n## Summary Count\n")
    md_content.append(f"Total Retained Rows: {len(df_filtered)}\n")

    md_content.append("\n## Variable Comparison\n")
    md_content.append("| Category | Variables | Reason |\n")
    md_content.append("|----------|-----------|--------|\n")
    md_content.append("| **Retained** | #run_no#, #Fraction_number#, #UV_1_280_mAU
        #, #UV_2_260_mAU#, #Cond_mS_cm#, #Conc_B_%#, #pH_pH# | >95% completeness
        and non-negative values |\n")
    md_content.append("| **Excluded** | #Conc_B_ml#, #pH_ml#, #UV_2_260_ml# |
        Flagged for removal per plan |\n")

    md_content.append("\n## Sample-Specific Modeling Linkage Rate\n")
    md_content.append(f"**Rate**: {linkage_rate:.2f}%\n")
    md_content.append(f"(Calculated as: {len(df_linkage)} rows with valid #
        chromatography_stage# and #Sample_Code# / {len(df_filtered)} total rows *
        100)\n")

    # Write to file
    output_filename = 'final_fit_for_use_summary.md'
    output_path = os.path.join(output_path, output_filename)

    with open(output_path, 'w') as f:
        f.write('\n'.join(md_content))

    print(f"Output saved to: {output_path}")

if __name__ == "__main__":
    Call_Code_Updates()
###
```

Listing 3: Code_3